Python

Từ thuật toán C++ đến chương trình dữ liệu chạy lại được

Phase 0 · Bài 0.1

Bảng điểm của nhóm lập trình

Ta cần đọc dữ liệu, gọi thư viện và lưu kết quả cho người khác chạy lại.

Vòng lặp, hàm, mảng, độ phức tạp: kiến thức nền đã có.

CÂU HỎI XUYÊN SUỐT

Làm sao viết chương trình ngắn mà kết quả vẫn đúng khi chạy lại từ đầu?

Mục tiêu buổi này

- Đọc đúng phép chia số âm và so sánh số thực.
- Phân biệt gán tên, sửa đối tượng và sao chép.
- Chọn cấu trúc dữ liệu, kiểm đáp án rồi đo tốc độ.
- Đọc CSV, đọc traceback và chạy notebook từ đầu.

Thụt lề thay cho dấu ngoặc

```
def passed_scores(values, threshold=5.0):
    result = []
    for value in values:
        if value >= threshold:
            result.append(value)
    return result
```

- Dấu `:` mở một khối.
- Các dòng trong cùng khối thụt lề như nhau.
- `return` ở ngoài `for`: thu đủ phần tử rồi trả về.
- `range(1, 5)` lấy 1, 2, 3, 4.

Notebook giữ trạng thái

Chỉ chạy lại ô đầu

```
[3]  n = 5

[2]  total = sum(range(n + 1))
```

6 cũ

Restart & Run All

```
[1]  n = 5

[2]  total = sum(range(n + 1))
```

15

Sửa n rồi chỉ chạy lại ô đầu: total vẫn giữ kết quả cũ.

Khởi động kernel mới, chạy tất cả ô từ trên xuống để kiểm lại.

HỎI NHANH

Vừa mở kernel mới, ta chạy ô `total = sum(range(n + 1))` **trước** ô `n = 3`. Ô ấy cho ra gì?

A `total` bằng 0, vì `n` chưa gán thì coi là 0

B `total` bằng 6, vì Python đọc hết notebook trước

C Lỗi `NameError`, vì chưa có tên `n`

D `total` dùng lại `n` của lần chạy trước

Chia số âm: khác C++

```
print(7 / 2, 6 / 2)
print(7 // 2, -7 // 2)
print(7 % 3, -7 % 3)
```

KẾT QUẢ

```
3.5 3.0
3 -4
1 2
```

// làm tròn xuống. Với $b \neq 0$:

$$a = (a//b) \cdot b + (a\%b).$$

`int` tăng độ chính xác theo nhu cầu; vẫn tốn thời gian và bộ nhớ khi số rất lớn.

Làm tròn xuống, không phải về 0



// làm tròn xuống; phép chia nguyên của C++ cắt về 0.

HỎI NHANH

`print(-7 // 2, -7 % 2)` in ra gì?

A `-3 -1`

B `-4 1`

C `-3 1`

D `-3.5 1`

Số thực cần kiểm sai số

```
import math

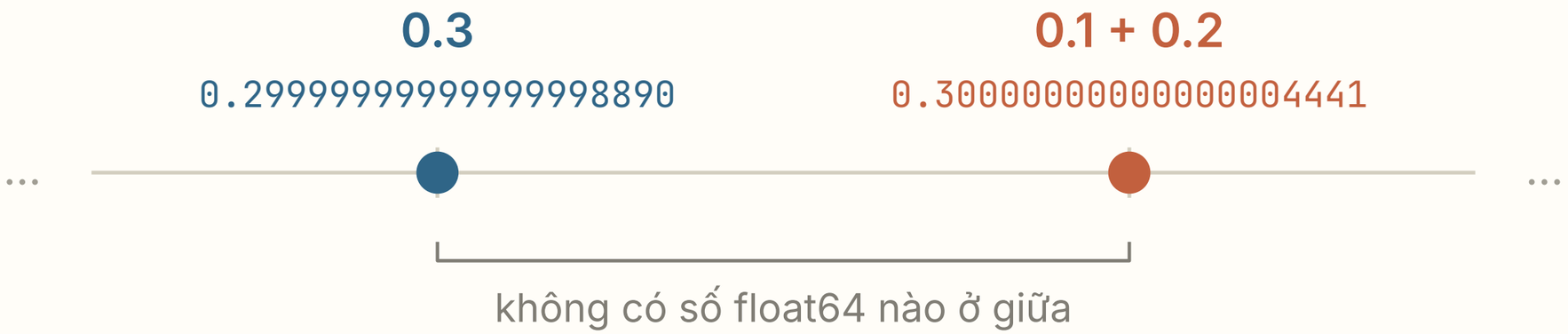
print(0.1 + 0.2, 0.1 + 0.2 == 0.3)
print(math.isclose(0.1 + 0.2, 0.3,
                   rel_tol=1e-9, abs_tol=1e-12))
print(round(2.5), round(3.5), round(0.125, 2))
```

KẾT QUẢ

```
0.30000000000000004 False
True
2 4 0.12
```

- Sai số nhị phân không phải lỗi thuật toán.
- Chọn tolerance theo đơn vị và độ chính xác cần kiểm.
- `round` chọn mốc chẵn khi đúng ở giữa.

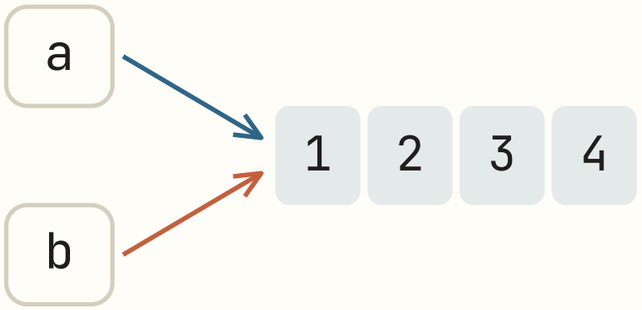
0.1 + 0.2 và 0.3 là hai số máy khác nhau



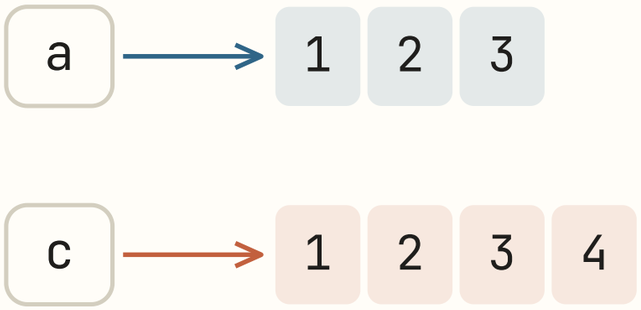
0.3 và 0.1 + 0.2 rơi vào hai số float64 liền kề.

Gán tên khác với sao chép

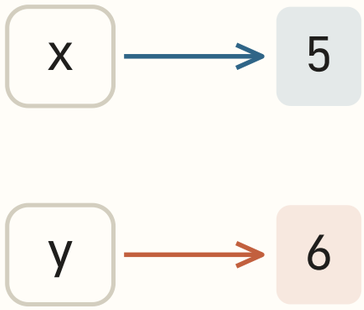
```
b = a
b.append(4)
```




```
c = a.copy()
c.append(4)
```




```
y = x
y += 1
```



Gán chỉ dán thêm tên; copy() tạo list mới; int bất biến.

Sửa list, hay gán lại tên?

```
a = [1, 2, 3]
b = a
b.append(4)
print(a, b, a is b)
```

KẾT QUẢ

```
[1, 2, 3, 4] [1, 2, 3, 4] True
```

- `append`: sửa nội dung đối tượng.
- `a = a + [5]`: tạo list mới, gán lại tên `a`.
- `is`: cùng đối tượng?
- `==`: giá trị bằng nhau?

HỎI NHANH

```
def reset(scores):
    scores.append(0)
    scores = []
marks = [7, 8]
reset(marks)
print(marks)
```

Đoạn code in ra gì?

A `[]`

B `[7, 8]`

C `[0]`

D `[7, 8, 0]`

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 94

List lồng nhau: một hàng hay ba hàng?

```
grid = [[0] * 3] * 3
grid[0][0] = 9
print(grid)

grid = [[0] * 3 for _ in range(3)]
grid[0][0] = 9
print(grid)
```

KẾT QUẢ

```
[[9, 0, 0], [9, 0, 0], [9, 0, 0]]
[[9, 0, 0], [0, 0, 0], [0, 0, 0]]
```

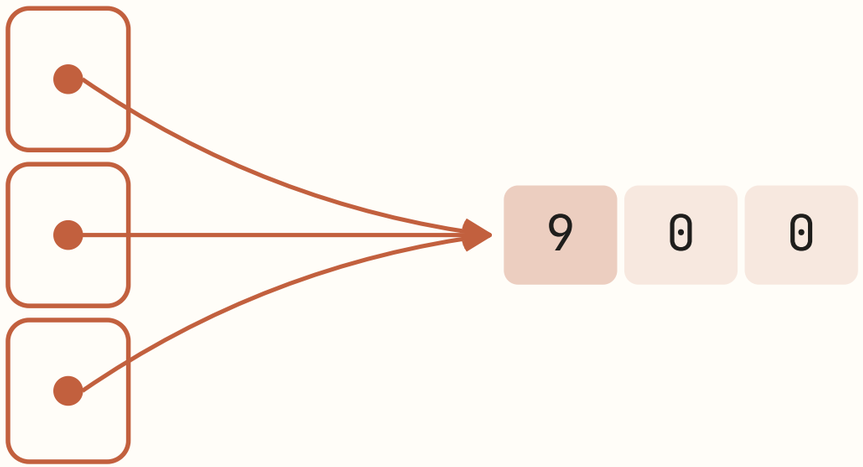
Nhân list ngoài lặp lại tham chiếu tới cùng hàng.

Comprehension tạo hàng mới ở mỗi lượt.

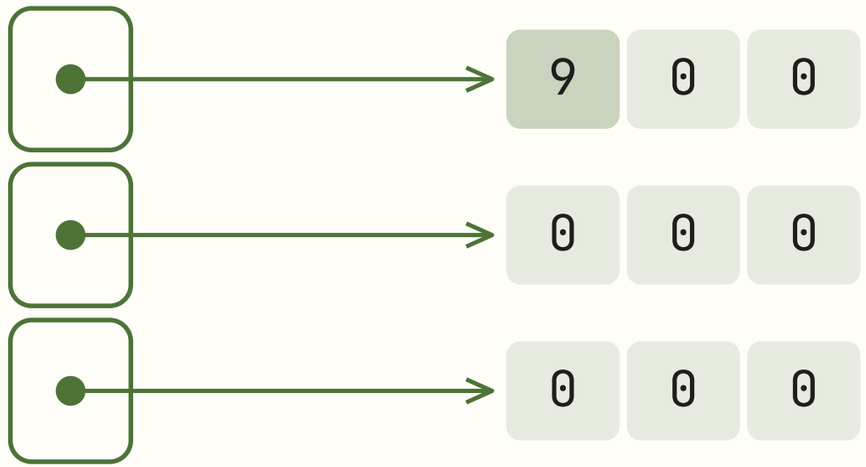
`copy()` một tầng; `deepcopy()` khi cần tách cấu trúc lồng nhau.

Ba tên, một hàng

```
[[0] * 3] * 3
```




```
[[0] * 3 for _ in range(3)]
```



Gán 9 vào ô đầu: bên trái cả ba hàng đổi, bên phải chỉ một hàng đổi.

HỎI NHANH

Chạy `a = [[1, 2], [3, 4]]`, `b = a.copy()`, `b[0].append(9)`. Lúc này `a` là gì?

A `[[1, 2], [3, 4]]`

B `[[1, 2, 9], [3, 4]]`

C `[[1, 2], [3, 4], 9]`

D Báo lỗi

Chọn cấu trúc như khi thi lập trình

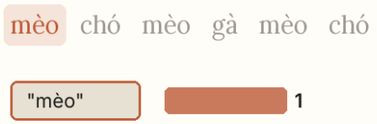
Công việc	Cấu trúc
Giữ thứ tự, lấy bằng vị trí	`list`
Một bộ giá trị cố định	`tuple`
Tra cứu theo khóa	`dict`
Kiểm thuộc tập nhiều lần	`set`

`x in list`: $O(n)$ ở trường hợp xấu.

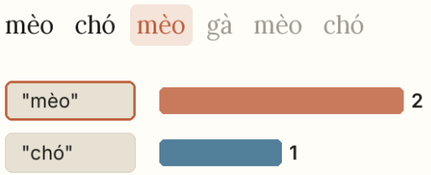
`x in set`: trung bình $O(1)$; tính cả chi phí dựng set.

Khóa cần **hashable** (có thể băm ổn định). `(3, 4)` được; tuple chứa list thì không.

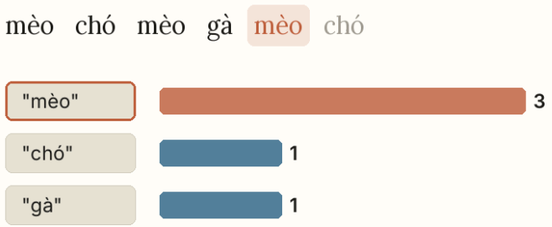
Đếm bằng dict



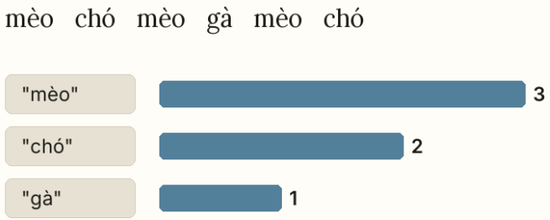
từ “mèo”: khóa mới, đặt bằng 1



từ “mèo”: đã có khóa, tăng 1 → 2



từ “mèo”: đã có khóa, tăng 2 → 3

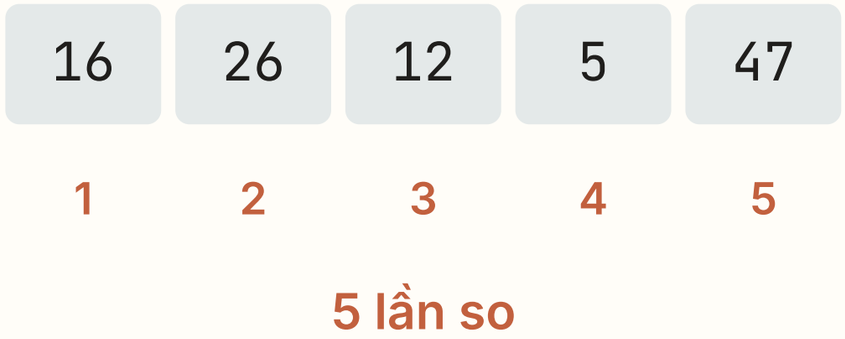


`most_common(2)` sắp theo số lần giảm dần: mèo 3, chó 2

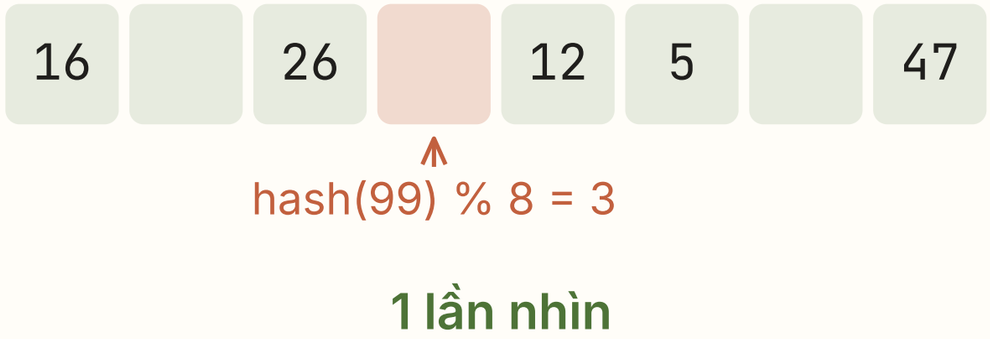
Đếm “mèo chó mèo gà mèo chó” bằng dict: mỗi từ chỉ chạm đúng một khóa, khóa mới bắt đầu từ 1.

Vì sao `in` trên set nhanh

`99 in list`



`99 in set`



List so từng phần tử; set nhảy thẳng tới một ngăn của bảng băm.

HỎI NHANH

Có 50 000 mã đã nộp bài. Cần kiểm 10 000 mã khác xem đã nộp chưa. Nên làm thế nào?

A Đổi danh sách thành `set` rồi dùng `in`

B Cứ dùng `in` trên list, 50 000 chưa nhiều

C Sắp xếp list trước cho `in` nhanh hơn

D Viết hai vòng `for` lồng nhau

Slicing, comprehension và vài hàm quen tay

```
a = [5, 3, 8, 1, 9, 2]
print(a[1:4], a[-1], a[-2:], a[::-1])
print([x * x for x in a if x % 2 == 1])
```

KẾT QUẢ

```
[3, 8, 1] 2 [9, 2] [2, 9, 1, 8, 3, 5]
[25, 9, 1, 81]
```

- `enumerate`: vị trí và phần tử.
- `zip`: phần tử cùng vị trí của nhiều dãy.
- `sorted(..., key=...)`: sắp theo một giá trị.
- `d.get(key, default)`: xử lý khóa chưa có.

Tham số mặc định được tạo một lần

```
def add_score(x, history=[]):
    history.append(x)
    return history

print(add_score(1), add_score(2))
```

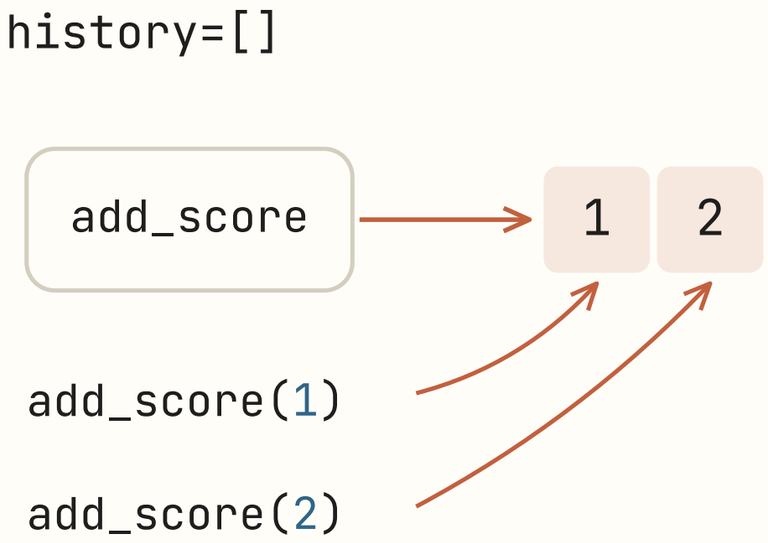
KẾT QUẢ

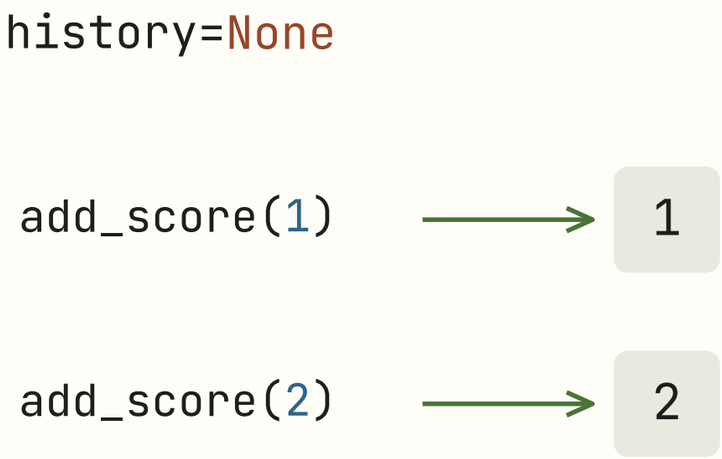
```
[1, 2] [1, 2]
```

Hai lần gọi dùng chung list mặc định.

```
def add_score(x, history=None):
    if history is None:
        history = []
    history.append(x)
    return history
```

Một list mặc định cho mọi lần gọi





List mặc định tạo một lần và dùng chung; None cho mỗi lần gọi một list mới.

HỎI NHANH

`add_score(score, history=[])` gọi lần sau vẫn thấy điểm của lần trước. Sửa thế nào để mỗi lần gọi bắt đầu từ list rỗng?

A Giữ `history=[]`, gọi `clear()` ở cuối hàm

B Giữ nguyên, vì Python tạo lại list mỗi lần

C Đặt `history=None`, tạo list mới trong hàm

D Dùng list toàn cục, gán `[]` trước mỗi lần

Một đáp án độc lập

Tổng $0 + 1 + \cdots + (n-1)$:

$$S = \frac{n(n-1)}{2}.$$

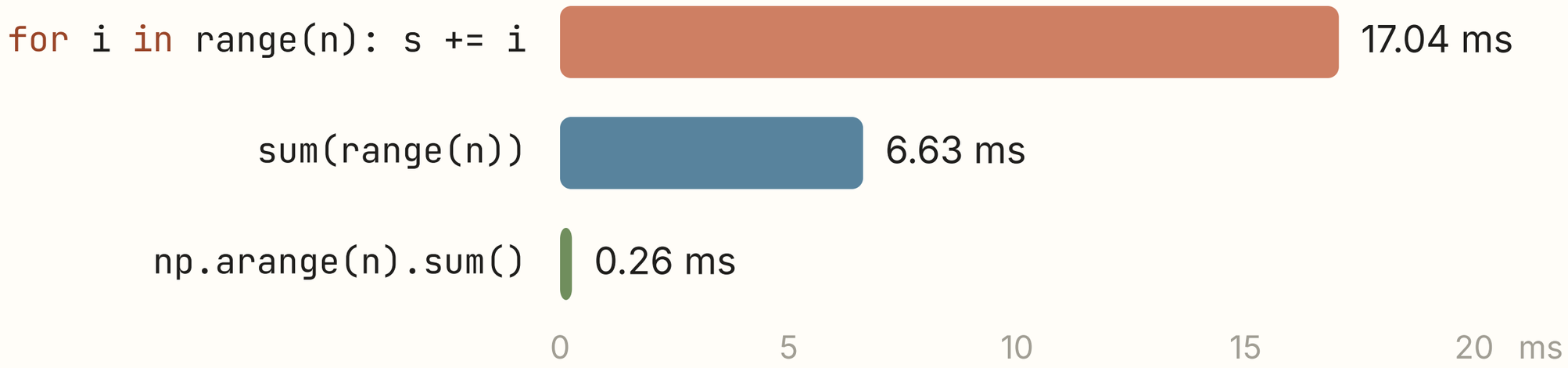
Với $n = 500\,000$: **124 999 750 000**.

Đối chiếu `loop_sum(n)` và `sum(range(n))` với công thức; NumPy cũng phải cho cùng đáp án.

Đo bằng `timeit.repeat`, cùng công việc, nhiều lần lặp.

Ghi rõ có tính bước tạo mảng hay không.

Bằng chứng của một công việc cụ thể



Cùng một tổng của 500 000 số, đo trên một máy CPU.

Đổi dữ liệu, thân vòng hay môi trường thì đo lại. Với cấp số cộng, công thức còn bỏ được cả vòng lặp.

HỎI NHANH

Hai cách đều tính đúng. Cách A chạy 8 ms nhưng phải chuẩn bị dữ liệu thêm 5 ms. Cách B chạy 11 ms, không cần chuẩn bị. Chỉ chạy một lần thì cách nào nhanh hơn?

A B, vì 11 ms ít hơn 13 ms

B Như nhau, vì cùng độ phức tạp

C Chưa so được, vì số đo dao động

D A, vì 8 ms ít hơn 11 ms

CSV phải được đọc lại

Giả sử `csv` đã được import, `path` là đường dẫn file cần ghi và `preds` là dict ánh xạ ID → dự đoán.

```
with path.open(
    "w", newline="", encoding="utf-8"
) as file:
    writer = csv.writer(file)
    writer.writerow(["id", "prediction"])
    for student_id, prediction in preds.items():
        writer.writerow(
            [student_id, prediction]
        )
```

- `with` đóng file khi ra khỏi khối.
- `encoding` và `newline` ghi rõ.
- Sau khi ghi, mở lại bằng `csv.reader` và kiểm header/số dòng.
- `csv.reader` đọc ô thành chuỗi; đổi kiểu trước khi tính.
- Module CSV xử lý cả dấu phẩy trong ô có ngoặc kép.

HỎI NHANH

Đọc cột điểm bằng `csv`, ta được `["9", "10", "2"]`. `sorted` trả về gì?

A `['2', '9', '10']`

B `['10', '2', '9']`

C `['9', '10', '2']`

D `TypeError`

Đọc traceback từ dưới lên

```
Traceback (most recent call last):
  File "…/diem.py", line 7, in <module>
    print(mean_of("Dũng"))
          ~~~~~~~^^^^^^^^
  File "…/diem.py", line 4, in mean_of
    marks = scores[name]
            ~~~~~~^^^^^^
KeyError: 'Dũng'
```

③ ai đã gọi

② ở dòng nào

① lỗi gì

đọc từ dưới lên

Traceback thật của Python 3.13: đọc từ dòng cuối lên.

Bốn lỗi hay gặp

Bốn thông báo cuối của bốn traceback riêng:

KẾT QUẢ

```
TypeError: can only concatenate str (not "int") to str
KeyError: 'Dũng'
IndexError: list index out of range
ValueError: invalid literal for int() with base 10: '7.5'
```

- Dòng cuối: loại lỗi và nguyên nhân.
- Các dòng `File ..., line ...`: vị trí cần mở code.
- Không báo lỗi: kiểm kiểu, độ dài, vài giá trị.
- `sum = 0` che hàm dựng sẵn; đặt `total`.

HỎI NHANH

`marks` có 3 phần tử. Vòng `for i in range(1, len(marks) + 1): print(marks[i])` chạy ra sao?

A In đủ ba phần tử như vòng 1 đến n của C++

B Báo `IndexError` ngay ở vòng đầu

C In `marks[1]`, `marks[2]` rồi dừng

D In `marks[1]`, `marks[2]`, rồi `IndexError`

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 137

Khi đi thi

- **Restart and run all** trước mỗi lần nộp.
- **Ghi rõ seed** cho mọi nguồn ngẫu nhiên.
- **Đúng trước, đo sau**; NumPy cho dữ liệu số lớn.
- Đường dẫn theo đề, `encoding="utf-8"`, đọc lại file nộp.

02

Luyện đề

LUYỆN ĐỀ 1

Sau `a = [1, 2]`, `b = [1, 2]`, `c = a`, lệnh `print(a == b, a is b, a is c)` in ra gì?

A `True True True`

B `True False True`

C `False False True`

D `True False False`

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 95

LUYỆN ĐỀ 2

Quen C++, một bạn viết `[1, 2] + [3, 4]` để cộng từng phần tử. Python cho kết quả gì?

A `[4, 6]`

B `[[1, 2], [3, 4]]`

C Báo lỗi

D `[1, 2, 3, 4]`

ĐỀ THI NOAI 2026 Singapore · Vòng sơ loại · Câu 91

LUYỆN ĐỀ 3

Cần đánh số các token khác nhau trong `["mèo", "chó", "mèo"]` thành 0, 1,... Chạy lại lần nào cũng phải ra đúng một bảng số. Viết thế nào?

A
```
{t: i for i, t in enumerate(sorted(set(tokens)))}
```

B
```
{t: i for i, t in enumerate(list(set(tokens)))}
```

C
```
{t: i for i, t in enumerate(sorted(tokens))}
```

D
```
{t: tokens.index(t) for t in tokens}
```

PHỎNG THEO USA-NA-AIO 2026 · Vòng 1 · Part 5.1–5.2

Chốt buổi học

Trước khi chuyển sang NumPy, kiểm bốn điều:

- tên không che hàm dựng sẵn;
- kiểu và phép chia đúng;
- notebook chạy từ kernel mới;
- CSV đã ghi được đọc lại.

Làm **10 câu cuối bài**; mỗi câu sai cần chỉ ra được lỗi và cách sửa.